

Phase 5 Structured Note: AVL Deletion, Height-Node Analysis, and Final Mastery

1. Where We Are Now

Before Phase 5, you already learned the main ideas of AVL trees.

From Phase 1

You learned that an AVL tree is a **Binary Search Tree**, or **BST**, with one extra rule:

Every node must stay balanced.

A BST follows this rule:

left subtree values < node value < right subtree values

An AVL tree also checks the **balance factor** of every node:

Balance Factor = height(left subtree) - height(right subtree)

Valid AVL balance factors are:

-1, 0, +1

Invalid balance factors are:

+2, -2, +3, -3, ...

So Phase 1 taught you how to **detect imbalance**.

From Phase 2

You learned that AVL trees fix imbalance using **rotations**.

The four insertion rotation cases are:

Case	Shape	Rotation type
LL	Left-left straight line	Single rotation
RR	Right-right straight line	Single rotation
LR	Left-right zig-zag	Double rotation
RL	Right-left zig-zag	Double rotation

So Phase 2 taught you how to **fix imbalance using rotations**.

From Phase 3

You learned how rotations work in C.

You learned that AVL insertion:

1. Inserts like a normal BST.
2. Returns upward through recursion.
3. Updates height.
4. Checks balance factor.
5. Applies a rotation if needed.
6. Returns the new root of the subtree.

So Phase 3 taught you how to implement AVL insertion safely with pointers.

From Phase 4

You traced a complete AVL insertion sequence:

10, 20, 30, 25, 28, 27, 5

You saw that AVL trees stay shorter than normal BSTs because they rotate when needed.

So Phase 4 taught you how AVL insertion works in a full example.

2. What Phase 5 Adds

Phase 5 teaches two final major topics:

1. **AVL deletion**
2. **Height-node analysis**

The big idea is:

Phase 5 explains how to delete from an AVL tree and why AVL trees are guaranteed to stay fast.

By the end of Phase 5, you should understand:

- AVL deletion
- BST deletion cases
- Leaf deletion
- One-child deletion
- Two-child deletion
- In-order predecessor
- In-order successor
- Height updates after deletion
- Balance checks after deletion
- Deletion rotation cases: L1, L-1, L0, R1, R-1, R0
- Why deletion rotations are still LL, LR, RR, and RL
- AVL deletion complexity
- Maximum nodes for a height
- Minimum nodes for a height
- Fibonacci-style recurrence
- Why AVL height is $O(\log N)$
- Why search, insertion, and deletion are all $O(\log N)$

Memory line:

AVL deletion = BST deletion + bottom-up rebalancing.

Part A: AVL Deletion

3. Big Idea of AVL Deletion

AVL deletion starts exactly like normal BST deletion.

Then, after the node is removed, AVL does extra work:

1. Update height.
2. Check balance factor.
3. Rotate if needed.

So the full idea is:

AVL deletion = normal BST deletion + height update + rebalancing

This is similar to insertion.

Insertion starts like BST insertion, then balances.

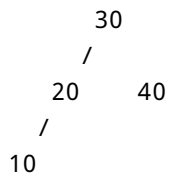
Deletion starts like BST deletion, then balances.

4. Why Deletion Can Make an AVL Tree Unbalanced

When we delete a node, one subtree may become shorter.

If one side becomes too short, the other side may become too tall in comparison.

Example before deletion:

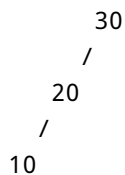


For node 30:

```
left height = 2
right height = 1
BF = 2 - 1 = +1
```

So node 30 is balanced.

Now delete 40:



Now for node 30:

```
left height = 2
right height = 0
BF = 2 - 0 = +2
```

BF = +2 is invalid.

So node 30 must be fixed with a rotation.

After rotation:

```
    20
   /
  10  30
```

Now the tree is balanced again.

5. Deletion Happens Going Down, Repair Happens Coming Up

This is one of the most important ideas in AVL deletion.

When deleting, the program first goes down the tree to find the node.

Then, after deleting it, the program returns back upward.

During this upward return, it updates heights and fixes imbalance.

Memory line:

```
Deletion happens while going down.
AVL repair happens while coming back up.
```

6. Normal BST Deletion Cases

Before learning AVL deletion, you must understand normal BST deletion.

There are three main deletion cases:

1. Delete a leaf node.
2. Delete a node with one child.

3. Delete a node with two children.

AVL deletion uses these same three cases first.

7. Case 1: Deleting a Leaf Node

A **leaf node** is a node with no children.

Example:

```
  20
 /
10
```

Delete 10.

Node 10 has:

```
left child = NULL
right child = NULL
```

So we simply remove it.

Result:

```
20
```

Simple idea:

```
If the node has no children, remove it directly.
```

8. Case 2: Deleting a Node with One Child

Example:

```
  20
 /
10
```

15

Delete 10.

Node 10 has one child: 15.

So we connect the parent of 10 directly to 15.

Result:

```
  20
 /
15
```

Simple idea:

If the node has one child, replace the node with its child.

Another way to imagine it:

Remove the node and connect around it.

9. Case 3: Deleting a Node with Two Children

This is the most important deletion case.

Example:

```
    30
   /
  20  40
 /
10  25
```

Delete 30.

Node 30 has two children:

```
left child = 20
right child = 40
```

We cannot simply remove 30, because then the left and right subtrees would lose their connection.

So we replace 30 with another value that keeps the BST order correct.

We can use either:

1. The **in-order predecessor**
2. The **in-order successor**

10. In-Order Predecessor

The **in-order predecessor** is the largest value in the left subtree.

To find the predecessor:

1. Go to the left child.
2. Then keep going right as far as possible.

Example:

```
      30
     /
    20  40
   /
  10  25
```

The left subtree of 30 is:

```
    20
   /
  10  25
```

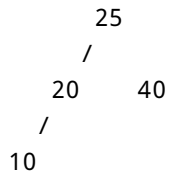
The largest value there is 25.

So:

```
predecessor of 30 = 25
```


If we replace 30 with 25, the BST order still works.

Result:



Simple memory line:

Predecessor = biggest value on the left side.

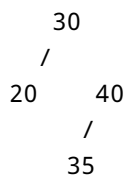
11. In-Order Successor

The **in-order successor** is the smallest value in the right subtree.

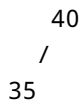
To find the successor:

1. Go to the right child.
2. Then keep going left as far as possible.

Example:



The right subtree of 30 is:



The smallest value there is 35.

So:

successor of 30 = 35

Simple memory line:

Successor = smallest value on the right side.

12. Predecessor vs Successor

When deleting a node with two children, either predecessor or successor can be used.

Replacement choice	Meaning
In-order predecessor	Biggest value in the left subtree
In-order successor	Smallest value in the right subtree

Both preserve the BST rule.

The BST rule is still:

left < node < right

Some implementations choose predecessor.

Some implementations choose successor.

Some choose based on subtree height.

For example:

If left subtree is taller, use predecessor.
If right subtree is taller or equal, use successor.

This can help keep the tree more balanced.

13. AVL Work After Normal Deletion

After normal BST deletion, AVL must do this while returning upward:

1. Update the current node height.
2. Calculate the current node balance factor.
3. If the balance factor is valid, keep returning upward.
4. If the balance factor is `+2` or `-2`, apply a rotation.
5. Return the new root of the repaired subtree.

The important rule is:

Always update height before checking balance factor.

Why?

Because balance factor depends on height.

If height is old, the balance factor may be wrong.

14. Balance Factor Reminder

The balance factor formula is:

$BF = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$

Meaning:

Balance factor	Meaning	AVL status
<code>+1</code>	Left side is taller by 1	Valid
<code>0</code>	Both sides have same height	Valid
<code>-1</code>	Right side is taller by 1	Valid
<code>+2</code>	Left side is too tall	Invalid
<code>-2</code>	Right side is too tall	Invalid

In deletion, we repair when we see:

BF = +2 or BF = -2

Part B: Deletion Rotation Cases

15. Why Deletion Rotation Names Look Different

During insertion, we used these names:

LL, RR, LR, RL

During deletion, we may see these names:

L1, L-1, L0, R1, R-1, R0

These names look new, but they do not mean new rotations.

They are only labels that describe:

1. Which side is too heavy.
2. What the child balance factor is.

The actual rotations are still:

LL, LR, RR, RL

Memory line:

Deletion names are labels. Rotations are still LL, LR, RR, and RL.

16. Meaning of L and R in Deletion Cases

L cases

An **L case** happens when the parent becomes left-heavy.

That means:

Parent BF = +2

The left side is too tall compared to the right side.

This often happens because deletion happened in the right subtree.

R cases

An **R case** happens when the parent becomes right-heavy.

That means:

Parent BF = -2

The right side is too tall compared to the left side.

This often happens because deletion happened in the left subtree.

17. Deletion Rotation Table

Deletion case	Parent BF	Child BF	Standard rotation
L1	+2	+1	LL
L-1	+2	-1	LR
L0	+2	0	LL or LR
R1	-2	-1	RR
R-1	-2	+1	RL
R0	-2	0	RR or RL

Beginner-friendly version:

Parent BF = +2 → inspect the left child.
Parent BF = -2 → inspect the right child.

18. L1 Case

When L1 happens

L1 happens when:

```
Parent BF = +2  
Left child BF = +1
```

That means:

```
The parent is too heavy on the left.  
The left child also leans left.
```

So the shape is a left-left problem.

Use an **LL rotation**.

Example

```
    30  
   /\n  20  
 /\n10
```

For node **30**:

```
BF(30) = +2
```

For node **20**:

```
BF(20) = +1
```

So this is L1.

Fix

Apply LL rotation.

Result:

```

      20
     /
    10  30

```

Memory line:

L1 = left-heavy parent + left-heavy child = LL rotation.

19. L-1 Case

When L-1 happens

L-1 happens when:

Parent BF = +2
Left child BF = -1

That means:

The parent is too heavy on the left.
The left child leans right.

So the shape is a left-right problem.

Use an **LR rotation**.

Example

```

      30
     /
    10
   /
  20

```

For node 30:

BF(30) = +2

For node 10 :

$BF(10) = -1$

So this is L-1.

Fix

Apply LR rotation.

Result:

```
    20
   /
  10  30
```

Memory line:

L-1 = left-heavy parent + right-heavy left child = LR rotation.

20. L0 Case

When L0 happens

L0 happens when:

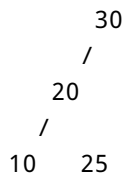
Parent BF = +2
Left child BF = 0

That means:

The parent is too heavy on the left.
The left child is perfectly balanced.

This case is more common in deletion than insertion.

Example



For node 30:

`BF(30) = +2`

For node 20:

`BF(20) = 0`

So this is L0.

Fix

For L0, either LL or LR can be used depending on the version of the algorithm.

In beginner C code, we usually choose LL because of this condition:

```
if (bf(p) == 2 && bf(p->left) >= 0)
    return ll(p);
```

Since `bf(p->left) == 0`, it satisfies:

`bf(p->left) >= 0`

So the code chooses LL.

Memory line:

L0 = left-heavy parent + balanced left child.
Most beginner code handles it with LL.

Important note:

After an L0 rotation, the new local root may not have balance factor `0`, but it will still be valid.

Valid means:

-1, 0, or +1

21. R1 Case

When R1 happens

R1 happens when:

Parent BF = -2
Right child BF = -1

That means:

The parent is too heavy on the right.
The right child also leans right.

So the shape is a right-right problem.

Use an **RR rotation**.

Example

30

40

50

For node `30`:

BF(30) = -2

For node `40`:

$BF(40) = -1$

So this is R1.

Fix

Apply RR rotation.

Result:

```
    40
   /
  30  50
```

Memory line:

R1 = right-heavy parent + right-heavy child = RR rotation.

22. R-1 Case

When R-1 happens

R-1 happens when:

Parent BF = -2
Right child BF = +1

That means:

The parent is too heavy on the right.
The right child leans left.

So the shape is a right-left problem.

Use an **RL rotation**.

Example

```
30
  /
 50
 /
40
```

For node 30 :

$BF(30) = -2$

For node 50 :

$BF(50) = +1$

So this is R-1.

Fix

Apply RL rotation.

Result:

```
  40
 /
30  50
```

Memory line:

R-1 = right-heavy parent + left-heavy right child = RL rotation.

23. R0 Case

When R0 happens

R0 happens when:

```
Parent BF = -2
Right child BF = 0
```

That means:

```
The parent is too heavy on the right.
The right child is perfectly balanced.
```

This is the mirror of L0.

Example

```
30
  40
 /
35 50
```

For node 30:

```
BF(30) = -2
```

For node 40:

```
BF(40) = 0
```

So this is R0.

Fix

For R0, either RR or RL can be used depending on the version of the algorithm.

In beginner C code, we usually choose RR because of this condition:

```
if (bf(p) == -2 && bf(p->right) <= 0)
    return rr(p);
```

Since `bf(p->right) == 0`, it satisfies:

```
bf(p->right) <= 0
```

So the code chooses RR.

Memory line:

R0 = right-heavy parent + balanced right child.
Most beginner code handles it with RR.

24. Beginner Decision Rules for Deletion Rotations

Use these rules in code:

```
If BF(parent) = +2 and BF(left child) >= 0 → LL
If BF(parent) = +2 and BF(left child) < 0 → LR

If BF(parent) = -2 and BF(right child) <= 0 → RR
If BF(parent) = -2 and BF(right child) > 0 → RL
```

This handles:

L1 and L0 with LL
L-1 with LR
R1 and R0 with RR
R-1 with RL

25. Deletion Rotation Summary

Parent balance	Child balance	Deletion name	Rotation
+2	left child +1	L1	LL
+2	left child 0	L0	LL in beginner code
+2	left child -1	L-1	LR
-2	right child -1	R1	RR
-2	right child 0	R0	RR in beginner code

Parent balance	Child balance	Deletion name	Rotation
-2	right child +1	R-1	RL

Memory line:

For deletion, inspect parent BF first, then inspect the taller child BF.

Part C: AVL Deletion Algorithm

26. AVL Deletion in Simple Steps

Here is the complete beginner algorithm:

```
deleteNode(root, key):
```

1. Search for the key like a normal BST.
2. If the key is smaller than the current node:
go left.
3. If the key is greater than the current node:
go right.
4. If the key is found:
delete it using normal BST deletion:
 - leaf case
 - one-child case
 - two-child case
5. After deletion, return upward.
6. At each ancestor:
update height.
7. Calculate balance factor.
8. If the node is imbalanced:
apply LL, LR, RR, or RL rotation.
9. Return the new root of the subtree.

27. Conceptual Pseudocode

```
Delete(p, key):  
  
    if p is NULL:  
        return NULL  
  
    if key < p->data:  
        p->left = Delete(p->left, key)  
  
    else if key > p->data:  
        p->right = Delete(p->right, key)  
  
    else:  
        delete this node using BST deletion rules  
  
    if p is NULL:  
        return NULL  
  
    update p->height  
  
    if p is unbalanced:  
        rotate  
  
    return new root of this subtree
```

Important idea:

The recursive result must be assigned back to the child pointer.

Example:

```
p->left = deleteNode(p->left, key);
```

Why?

Because deletion or rotation may change the root of that subtree.

28. Rebalance Function

A useful way to write AVL code is to put the balancing logic in one function.


```

Node *rebalance(Node *p) {
    update(p);

    if (bf(p) == 2 && bf(p->left) >= 0)
        return ll(p);

    if (bf(p) == 2 && bf(p->left) < 0)
        return lr(p);

    if (bf(p) == -2 && bf(p->right) <= 0)
        return rr(p);

    if (bf(p) == -2 && bf(p->right) > 0)
        return rl(p);

    return p;
}

```

This works because:

- `bf(p) == 2` means the left side is too tall.
- `bf(p) == -2` means the right side is too tall.
- The child balance factor tells us which rotation is needed.

29. Why Deletion Is Sometimes Harder Than Insertion

Insertion usually makes a subtree taller.

Deletion usually makes a subtree shorter.

After deletion, the height decrease can continue upward.

That means several ancestors may need to be checked.

So after deletion, we keep returning upward and checking every ancestor on the path.

Memory line:

Deletion can shrink a subtree, so keep checking upward.

30. Complete C Program for AVL Insertion, Deletion, and Height Math

This program includes:

- AVL node structure
- Height function
- Balance factor function
- Four rotations
- Rebalance function
- Insert function
- Delete function
- Predecessor function
- Successor function
- Minimum nodes for height
- Maximum nodes for height
- Inorder traversal

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    int height;
    struct Node *left;
    struct Node *right;
} Node;

int max(int a, int b) {
    return (a > b) ? a : b;
}

int h(Node *p) {
    return p ? p->height : 0;
}

int bf(Node *p) {
    return p ? h(p->left) - h(p->right) : 0;
}

Node *newNode(int key) {
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = key;
    p->height = 1;
    p->left = NULL;
    p->right = NULL;
}
```

```

        return p;
    }

    void update(Node *p) {
        if (p != NULL) {
            p->height = 1 + max(h(p->left), h(p->right));
        }
    }

    Node *ll(Node *p) {
        Node *pl = p->left;
        Node *plr = pl->right;

        pl->right = p;
        p->left = plr;

        update(p);
        update(pl);

        return pl;
    }

    Node *rr(Node *p) {
        Node *pr = p->right;
        Node *prl = pr->left;

        pr->left = p;
        p->right = prl;

        update(p);
        update(pr);

        return pr;
    }

    Node *lr(Node *p) {
        Node *pl = p->left;
        Node *plr = pl->right;

        pl->right = plr->left;
        p->left = plr->right;

        plr->left = pl;
        plr->right = p;

        update(pl);
        update(p);
        update(plr);
    }

```

```

        return plr;
    }

Node *rl(Node *p) {
    Node *pr = p->right;
    Node *prl = pr->left;

    pr->left = prl->right;
    p->right = prl->left;

    prl->right = pr;
    prl->left = p;

    update(pr);
    update(p);
    update(prl);

    return prl;
}

Node *rebalance(Node *p) {
    if (p == NULL) {
        return NULL;
    }

    update(p);

    if (bf(p) == 2 && bf(p->left) >= 0) {
        return ll(p);
    }

    if (bf(p) == 2 && bf(p->left) < 0) {
        return lr(p);
    }

    if (bf(p) == -2 && bf(p->right) <= 0) {
        return rr(p);
    }

    if (bf(p) == -2 && bf(p->right) > 0) {
        return rl(p);
    }

    return p;
}

Node *insert(Node *p, int key) {

```

```

    if (p == NULL) {
        return newNode(key);
    }

    if (key < p->data) {
        p->left = insert(p->left, key);
    } else if (key > p->data) {
        p->right = insert(p->right, key);
    } else {
        return p;
    }

    return rebalance(p);
}

Node *inPre(Node *p) {
    while (p != NULL && p->right != NULL) {
        p = p->right;
    }
    return p;
}

Node *inSucc(Node *p) {
    while (p != NULL && p->left != NULL) {
        p = p->left;
    }
    return p;
}

Node *deleteNode(Node *p, int key) {
    if (p == NULL) {
        return NULL;
    }

    if (key < p->data) {
        p->left = deleteNode(p->left, key);
    } else if (key > p->data) {
        p->right = deleteNode(p->right, key);
    } else {
        if (p->left == NULL && p->right == NULL) {
            free(p);
            return NULL;
        } else if (p->left == NULL) {
            Node *child = p->right;
            free(p);
            return child;
        } else if (p->right == NULL) {
            Node *child = p->left;

```

```

        free(p);
        return child;
    } else {
        Node *q;

        if (h(p->left) > h(p->right)) {
            q = inPre(p->left);
            p->data = q->data;
            p->left = deleteNode(p->left, q->data);
        } else {
            q = inSucc(p->right);
            p->data = q->data;
            p->right = deleteNode(p->right, q->data);
        }
    }
}

return rebalance(p);
}

long long minNodesForHeight(int height) {
    if (height <= 0) {
        return 0;
    }

    if (height == 1) {
        return 1;
    }

    return minNodesForHeight(height - 1) +
        minNodesForHeight(height - 2) +
        1;
}

long long maxNodesForHeight(int height) {
    long long result = 1;

    for (int i = 0; i < height; i++) {
        result *= 2;
    }

    return result - 1;
}

void inorder(Node *p) {
    if (p == NULL) {
        return;
    }
}

```

```

        inorder(p->left);
        printf("%d(h=%d,bf=%d) ", p->data, p->height, bf(p));
        inorder(p->right);
    }

    int main(void) {
        int keys[] = {30, 20, 40, 10, 25, 35, 50, 5, 15};
        int n = sizeof(keys) / sizeof(keys[0]);

        Node *root = NULL;

        for (int i = 0; i < n; i++) {
            root = insert(root, keys[i]);
        }

        printf("Before deletion: ");
        inorder(root);
        printf("\n");

        root = deleteNode(root, 50);

        printf("After deleting 50: ");
        inorder(root);
        printf("\n");

        for (int height = 1; height <= 7; height++) {
            printf("height %d: min nodes=%lld, max nodes=%lld\n",
                height,
                minNodesForHeight(height),
                maxNodesForHeight(height));
        }

        return 0;
    }

```

31. Explanation of the C Program

Node structure

```

typedef struct Node {
    int data;
    int height;
    struct Node *left;

```

```
    struct Node *right;
} Node;
```

Each node stores:

Field	Meaning
data	The value stored in the node
height	Height of the subtree rooted at this node
left	Pointer to the left child
right	Pointer to the right child

Height function

```
int h(Node *p) {
    return p ? p->height : 0;
}
```

Meaning:

If p exists, return p->height.
If p is NULL, return 0.

This matches the rule:

NULL child height = 0

Balance factor function

```
int bf(Node *p) {
    return p ? h(p->left) - h(p->right) : 0;
}
```

This follows the formula:


```
BF = height(left subtree) - height(right subtree)
```

Update function

```
void update(Node *p) {  
    if (p != NULL) {  
        p->height = 1 + max(h(p->left), h(p->right));  
    }  
}
```

This follows the height formula:

```
height = 1 + max(left height, right height)
```

Rotation functions

The program uses the same four rotations you learned earlier:

Function	Rotation
ll	LL rotation
rr	RR rotation
lr	LR rotation
rl	RL rotation

Each rotation:

1. Saves important pointers.
2. Rearranges child links.
3. Updates heights.
4. Returns the new root of the subtree.

Rebalance function

```
Node *rebalance(Node *p)
```

This function checks balance and chooses the correct rotation.

It handles both insertion and deletion.

Insert function

```
Node *insert(Node *p, int key)
```

This function:

1. Inserts like a normal BST.
 2. Calls `rebalance(p)` while returning upward.
-

Predecessor function

```
Node *inPre(Node *p)
```

This finds the biggest value in the left subtree.

It keeps moving right:

```
while (p && p->right) p = p->right;
```

Successor function

```
Node *inSucc(Node *p)
```

This finds the smallest value in the right subtree.

It keeps moving left:

```
while (p && p->left) p = p->left;
```

Delete function

```
Node *deleteNode(Node *p, int key)
```

This function:

1. Searches for the key.
2. Deletes the node using BST deletion rules.
3. Uses predecessor or successor if the node has two children.
4. Calls `rebalance(p)` while returning upward.

32. Program Dry Run in Simple Words

The program inserts:

```
30, 20, 40, 10, 25, 35, 50, 5, 15
```

Then it prints the tree in inorder order.

Because AVL is still a BST, inorder traversal prints values in sorted order.

Then the program deletes:

```
50
```

After deleting `50`, the program returns upward and checks whether any ancestor became unbalanced.

If an ancestor is too left-heavy or too right-heavy, the `rebalance` function rotates.

Then the program prints the tree again.

Finally, it prints:

```
minimum nodes for each height  
maximum nodes for each height
```

for heights `1` through `7`.

Part D: Complexity of AVL Deletion

33. Cost of Searching for the Node

To delete a key, we first need to find it.

In an AVL tree, height is $O(\log N)$.

So searching for the node takes:

$O(\log N)$

34. Cost of Deleting the Node

Once the node is found:

- If it is a leaf, deletion is simple.
- If it has one child, replacement is simple.
- If it has two children, we find predecessor or successor along a subtree path.

This work is still bounded by the height of the tree.

Since AVL height is $O(\log N)$, this part is also within:

$O(\log N)$

35. Cost of Updating Heights

After deletion, we update height while returning upward.

We only return along one path from the deleted node back to the root.

That path has length:

$O(\log N)$

At each node, height update is constant time:

$O(1)$

So total height-update work is:

$O(\log N)$

36. Cost of Rotations

Each rotation changes only a fixed number of pointers.

So one rotation takes:

$O(1)$

Rotations do not scan the entire tree.

They only repair a small local area.

Memory line:

A rotation is $O(1)$ because it does fixed pointer work.

37. Total AVL Deletion Complexity

Step	Time cost
Search for key	$O(\log N)$
Delete like BST	part of the same path
Update heights upward	$O(\log N)$
Rotations	$O(1)$ per local repair
Total	$O(\log N)$

So:

AVL deletion = $O(\log N)$

Recursive deletion uses stack space.

The recursion follows the height of the tree.

So space complexity is:

$O(\log N)$

Part E: Height-Node Analysis

38. Why We Study Height-Node Analysis

AVL operations are fast because the tree stays short.

Search, insertion, and deletion all follow paths in the tree.

So their cost depends on tree height.

If height is small, operations are fast.

If height is large, operations are slow.

The goal of height-node analysis is to prove:

AVL height = $O(\log N)$

Once we know that, we can say:

Search = $O(\log N)$
Insertion = $O(\log N)$
Deletion = $O(\log N)$

39. Maximum Nodes for a Given Height

The maximum number of nodes happens when the tree is completely full.

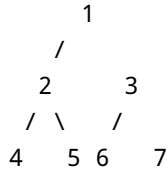
For height h , the maximum number of nodes is:

$$\text{MaxNodes} = 2^h - 1$$

Examples:

Height h	Maximum nodes $2^h - 1$
1	1
2	3
3	7
4	15
5	31
6	63
7	127

Example for height 3 :



This has:

$$2^3 - 1 = 7 \text{ nodes}$$

Memory line:

A full binary tree has $2^h - 1$ nodes.

40. Minimum Nodes for a Given AVL Height

Now we ask a different question:

What is the fewest number of nodes an AVL tree can have while still having height h ?

This is important because the tallest possible AVL tree for a given number of nodes is the sparsest valid AVL tree.

To be AVL-balanced, a node's two subtrees can differ in height by at most 1.

So for a tree of height h , the sparsest valid AVL shape has:

```
one subtree of height  $h - 1$   
one subtree of height  $h - 2$   
one root node
```

That gives the recurrence:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

Base cases:

$$\begin{aligned} N(0) &= 0 \\ N(1) &= 1 \end{aligned}$$

Meaning:

$$N(h) = \text{minimum number of nodes needed for AVL height } h$$

41. Why the Minimum-Node Formula Works

Suppose the AVL tree has height h .

That means one child subtree must have height:

$$h - 1$$

To use as few nodes as possible, the other child subtree should be as short as AVL allows.

AVL allows the two subtree heights to differ by at most 1.

So the other subtree can be:

$$h - 2$$

Then we add the root node:

$$+ 1$$

So:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

Memory line:

Tallest AVL with fewest nodes uses subtree heights $h-1$ and $h-2$.

42. Minimum Nodes Table

Using:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

with:

$$\begin{aligned} N(0) &= 0 \\ N(1) &= 1 \end{aligned}$$

we get:

Height h	Minimum nodes $N(h)$	Calculation
0	0	base case
1	1	base case
2	2	$N(1) + N(0) + 1 = 1 + 0 + 1$
3	4	$N(2) + N(1) + 1 = 2 + 1 + 1$
4	7	$N(3) + N(2) + 1 = 4 + 2 + 1$
5	12	$N(4) + N(3) + 1 = 7 + 4 + 1$

Height h	Minimum nodes $N(h)$	Calculation
6	20	$N(5) + N(4) + 1 = 12 + 7 + 1$
7	33	$N(6) + N(5) + 1 = 20 + 12 + 1$

The numbers grow quickly.

That is important because it means AVL height cannot grow too quickly.

43. Minimum Nodes vs Maximum Nodes

Height	Minimum AVL nodes	Maximum binary tree nodes
1	1	1
2	2	3
3	4	7
4	7	15
5	12	31
6	20	63
7	33	127

The maximum-node formula tells us the biggest possible full tree.

The minimum-node recurrence tells us the smallest possible AVL tree of that height.

Both help us understand the relationship between height and node count.

44. Fibonacci Connection

The AVL minimum-node recurrence is:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

The Fibonacci recurrence is:

$$F(h) = F(h - 1) + F(h - 2)$$

They look very similar.

The AVL recurrence has an extra `+1`, but the growth pattern is still Fibonacci-like.

Fibonacci numbers grow exponentially.

So AVL minimum nodes also grow exponentially with height.

That means:

nodes grow fast as height increases

So the reverse is:

height grows slowly as nodes increase

That slow growth is logarithmic.

So:

AVL height = $O(\log N)$

45. Why Fibonacci-Like Growth Proves $O(\log N)$

If the minimum number of nodes grows exponentially with height, then height grows logarithmically with the number of nodes.

Simple example:

Height increases slowly, but nodes increase very fast.

For AVL trees, the height is not allowed to become a long chain.

It must stay close to logarithmic.

That is why AVL trees can guarantee fast operations.

Memory line:

Minimum AVL nodes grow like Fibonacci, so AVL height is logarithmic.

46. Useful Height Estimate

A useful approximation for maximum AVL height is:

Maximum AVL height $\approx 1.44 \log_2(n + 2)$

In Big O notation, constants are ignored.

So:

$1.44 \log_2(n + 2)$ becomes $O(\log N)$

This is why we say:

AVL height = $O(\log N)$

47. Final Complexity of AVL Operations

Because AVL height is $O(\log N)$, major operations are also $O(\log N)$.

Operation	Time complexity	Why
Search	$O(\log N)$	Follow one path down the tree
Insertion	$O(\log N)$	Insert down one path, rebalance upward
Deletion	$O(\log N)$	Delete down one path, rebalance upward
One rotation	$O(1)$	Fixed number of pointer changes
Height update at one node	$O(1)$	Reads stored child heights
Balance factor at one node	$O(1)$	Subtracts stored child heights

Memory line:

AVL trees are fast because their height stays logarithmic.

Part F: Common Beginner Mistakes

48. Mistake 1: Deleting Like a BST but Forgetting to Rebalance

Wrong idea:

Once the BST deletion is done, the AVL deletion is done.

Correct idea:

After BST deletion, update height and check balance while returning upward.

49. Mistake 2: Thinking L1, L-1, L0, R1, R-1, R0 Are New Rotations

Wrong idea:

Deletion has six new rotations.

Correct idea:

Deletion has six labels, but the actual rotations are still LL, LR, RR, and RL.

50. Mistake 3: Ignoring the Two-Child Deletion Case

Wrong idea:

If a node has two children, just remove it.

Problem:

That breaks the BST links.

Correct idea:

Replace it with predecessor or successor, then delete that replacement from its old position.

51. Mistake 4: Forgetting to Return the New Subtree Root

After deletion or rotation, the root of a subtree may change.

Wrong idea:

Call delete or rotate, but ignore the returned pointer.

Correct idea:

```
p->left = deleteNode(p->left, key);  
p->right = deleteNode(p->right, key);  
root = deleteNode(root, key);
```

Memory line:

Always store the returned subtree root.

52. Mistake 5: Checking Balance Before Updating Height

Wrong order:

Check BF first.
Update height later.

Correct order:

Update height first.
Then check BF.

Why?

Because balance factor uses height.

53. Mistake 6: Thinking L0 or R0 Must Produce Perfect Balance

Wrong idea:

After L0 or R0, every balance factor must become 0.

Correct idea:

After L0 or R0, balance factors may be -1 or +1, and that is still valid.

Valid balance factors are:

-1, 0, +1

54. Mistake 7: Thinking $O(1)$ Rotations Make Deletion $O(1)$

Wrong idea:

Rotations are $O(1)$, so deletion is $O(1)$.

Correct idea:

The rotation is $O(1)$, but finding the node and returning upward take $O(\log N)$.

So total AVL deletion is:

$O(\log N)$

55. Mistake 8: Forgetting Why AVL Height Is Logarithmic

Wrong idea:

AVL is $O(\log N)$ just because we say so.

Correct idea:

AVL height is $O(\log N)$ because the minimum number of nodes grows in a Fibonacci-like way.

Part G: Quick Reference Tables

56. BST Deletion Cases

Case	Meaning	Action
Leaf node	No children	Remove directly
One child	One child exists	Replace node with child
Two children	Left and right children exist	Replace with predecessor or successor

57. Predecessor and Successor

Term	Meaning	How to find
In-order predecessor	Biggest value in left subtree	Go left, then all the way right
In-order successor	Smallest value in right subtree	Go right, then all the way left

58. Deletion Rotation Table

Deletion case	Parent BF	Child BF	Rotation
L1	+2	+1	LL
L-1	+2	-1	LR
L0	+2	0	LL or LR; beginner code usually LL
R1	-2	-1	RR
R-1	-2	+1	RL

Deletion case	Parent BF	Child BF	Rotation
R0	-2	0	RR or RL; beginner code usually RR

59. Beginner Code Decision Table

Code condition	Meaning	Rotation
<code>bf(p) == 2 && bf(p->left) >= 0</code>	Left-left or L0	LL
<code>bf(p) == 2 && bf(p->left) < 0</code>	Left-right	LR
<code>bf(p) == -2 && bf(p->right) <= 0</code>	Right-right or R0	RR
<code>bf(p) == -2 && bf(p->right) > 0</code>	Right-left	RL

60. Height-Node Formulas

Formula	Meaning
<code>MaxNodes = 2^h - 1</code>	Maximum nodes in a full binary tree of height <code>h</code>
<code>N(0) = 0</code>	Empty tree has height 0 and 0 nodes
<code>N(1) = 1</code>	One-node tree has height 1 and 1 node
<code>N(h) = N(h - 1) + N(h - 2) + 1</code>	Minimum nodes in AVL tree of height <code>h</code>
<code>height = O(log N)</code>	AVL height grows logarithmically

Part H: Practice Questions

61. Practice Question 1

What is AVL deletion?

Answer:

AVL deletion is normal BST deletion followed by height update and rebalancing.

62. Practice Question 2

What are the three BST deletion cases?

Answer:

1. Delete a leaf node.
2. Delete a node with one child.
3. Delete a node with two children.

63. Practice Question 3

What is the in-order predecessor?

Answer:

The biggest value in the left subtree.

64. Practice Question 4

What is the in-order successor?

Answer:

The smallest value in the right subtree.

65. Practice Question 5

If parent BF is `+2` and left child BF is `+1`, what deletion case is it?

Answer:

L1

Rotation:

LL rotation

66. Practice Question 6

If parent BF is $+2$ and left child BF is -1 , what deletion case is it?

Answer:

L-1

Rotation:

LR rotation

67. Practice Question 7

If parent BF is -2 and right child BF is -1 , what deletion case is it?

Answer:

R1

Rotation:

RR rotation

68. Practice Question 8

If parent BF is -2 and right child BF is $+1$, what deletion case is it?

Answer:

R-1

Rotation:

RL rotation

69. Practice Question 9

What is special about L0 and R0?

Answer:

The child balance factor is 0. These cases are common in deletion. Either rotation variant can work, but beginner code usually chooses LL for L0 and RR for R0.

70. Practice Question 10

What is the time complexity of AVL deletion?

Answer:

$O(\log N)$

Why?

Because AVL height is $O(\log N)$, and deletion follows one path down and one path back up.

71. Practice Question 11

What is the maximum number of nodes in a binary tree of height h ?

Answer:

$2^h - 1$

72. Practice Question 12

What is the minimum-node recurrence for AVL height?

Answer:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

with:

$$\begin{aligned} N(0) &= 0 \\ N(1) &= 1 \end{aligned}$$

73. Practice Question 13

Why is the AVL recurrence connected to Fibonacci numbers?

Answer:

Because it uses the previous two height values, similar to the Fibonacci recurrence.

74. Practice Question 14

Why are AVL operations $O(\log N)$?

Answer:

Because AVL trees keep their height logarithmic, and operations follow the height of the tree.

Part I: Phase 5 Final Summary

75. Main Ideas

Phase 5 completes the AVL tree topic.

The first main idea is deletion:

AVL deletion starts as normal BST deletion.

Then AVL adds:

height update + balance check + rotation

The second main idea is height analysis:

AVL trees stay fast because their height stays $O(\log N)$.

76. Final Memory Lines

AVL deletion = BST deletion + bottom-up rebalancing.

Deletion happens going down, but repair happens coming back up.

Deletion cases L1, L-1, L0, R1, R-1, R0 are labels, not new rotations.

The real rotations are still LL, LR, RR, and RL.

Predecessor = biggest value on the left.

Successor = smallest value on the right.

Maximum nodes for height $h = 2^h - 1$.

Minimum AVL nodes follow $N(h) = N(h-1) + N(h-2) + 1$.

The recurrence is Fibonacci-like, so AVL height is $O(\log N)$.

AVL search, insertion, and deletion are all $O(\log N)$.

Part J: Phase 5 Mastery Checklist

You understand Phase 5 if you can:

- Explain that AVL deletion starts like BST deletion.
- Explain why AVL deletion must rebalance afterward.
- Delete a leaf node.
- Delete a node with one child.
- Delete a node with two children.
- Explain in-order predecessor.
- Explain in-order successor.
- Explain why height updates happen while returning upward.
- Explain why balance factor must be checked after deletion.
- Identify L1.
- Identify L-1.
- Identify L0.
- Identify R1.
- Identify R-1.
- Identify R0.
- Translate deletion cases into LL, LR, RR, and RL rotations.
- Explain why L0 and R0 are special deletion cases.
- Explain why a rotation is $O(1)$.
- Explain why AVL deletion is $O(\log N)$.
- Use the maximum-node formula $2^h - 1$.
- Use the minimum-node recurrence $N(h) = N(h - 1) + N(h - 2) + 1$.
- Explain the Fibonacci connection.
- Explain why AVL height is $O(\log N)$.
- Explain why AVL search, insertion, and deletion are guaranteed $O(\log N)$.

Final One-Sentence Memory Line

Phase 5 teaches that AVL trees delete like BSTs, rebalance on the way back up, and stay fast because their height grows only logarithmically.